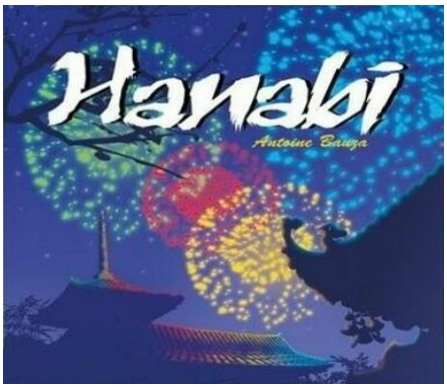
Suivi des connaissances - Projet Multi-Agents

Tom David, Pasquier Titouan, Guillais Marius, Chepy Maxim

Semestre 1 - 2025/2026





Sommaire

1 Description Générale **3**
1.1 Système Multi-Agents 3
1.2 Objectifs du projet 3
1.3 Organistation des fichiers 3

2 Agents **4**
2.1 Fonctionnement 4
2.2 Implémentation 4

3 Implémentations **5**
3.1 Hanabi 5
3.1.1 Principe 5
3.1.2 Implémentation 6
3.2 Jeu de la ville 6
3.2.1 Principe 6
3.2.2 Implémentation 6

4 Conclusion **7**
4.1 Avancé actuelle 7
4.2 Répartition des tâches 7
4.3 Améliorations 8

1 Description Générale

1.1 Système Multi-Agents

Un système multi-agents est un système composé d'un ensemble d'agents actifs dans un certain environnement et interagissant selon certaines règles.

Ce type d'implémentations a pour objectifs de décentralisé la résolution de certains problèmes. Cette méthode a été inspiré notamment par les fourmis et les abeilles, elle repose sur le fait de divisé l'objectif final en tâches plus simples, réalisés par plusieurs agents autonomes.

Cette méthode a été utilisé dans de nombreux problème tel que le tri couvain, les problèmes d'explorations et de couverture notamment utilisé après des catastrophes naturelles ou encore de dilemme du prisonnier itéré.

1.2 Objectifs du projet

Pour ce projet, nous allons nous concentrer sur les systèmes multi-agents dans lesquelles les agents n'ont pas la possibilité de communiquer entre eux directement, les agents ne pourront se basé uniquement sur leurs observations de l'environnement et sur leur connaissance de la stratégie de l'autre agent.

L'objectif est de faire un modèle de ce type d'agent robuste et capable de s'adapter à n'importe quel modèle qui correspond à certaines conditions en java.

Pour ce faire, nous nous sommes concentré sur 2 implémentations : Hanabi et le jeu de la ville (problème du rendez-vous).

1.3 Organistation des fichiers

Le package "core" contient tous les fichiers permettant l'implémentation de l'agent.

L'implémentation de Hanabi se trouve dans le package "hanabi".

Enfin, dans le package "ville", on retrouve les fichiers relatifs au jeu de la ville.

Pour pouvoir lancer le projet, il y a 2 fichiers "run_ville2.sh" et "run_hanabi.sh" qui éxécute respectivement le jeu de la ville et hanabi.

2 Agents

2.1 Fonctionnement

Un agent doit être capable de s'adapter à n'importe quel environnement et de décider quel action choisir à chaque pas de temps, pour ce faire il doit pouvoir se basé sur ses propres observations ainsi que sur ses actions précédentes.

De plus, il doit pouvoir être utilisé dans des implémentations très diverses sans avoir besoin de le modifier et sans qu'il ne perde en efficacité.

2.2 Implémentation

Afin de répondre aux critères précédemment cités, nous avons d'abord cherché une modélisation générale qui permettrait de représenter n'importe quel environnement. Après réflexion rapide, nous nous sommes rendu compte que c'était impossible et en avons déduit qu'il fallait que la modélisation soit fourni au cas par cas.

Nous avons d'ailleurs eut la même réflexion pour la stratégie utilisée, ce que nous allons également appelé **politique**.

Pour ce qui est du code, nous avons créer une classe **Agent** qui prend un historique et une politique en argument, une **politique** étant une interface dont la réalisation concrète doit être fournie par l'implémentation. Pour **l'historique**, c'est une Map d'**actions** et d'**observation** dont ces derniers seront récupérés dans l'implémentation.

Nous avons également créer une politique par défaut, qui s'appelle politique random et qui fait une aléatoire à chaque pas de temps.

Nous avons eu besoin de faire d'autres implémentations telle que **ActionJointe** qui est une Map d'agent et d'actions et qui associe à chaque agent l'action qu'il a réalisé pour chaque pas de temps.

Mais aussi un **simulateur** qui prend un jeu ainsi qu'une Map d'agent et de politique, et qui va s'occuper, dans un premier temps, d'utiliser la politique associé à chaque agent afin de récupérer les actions qui vont être utilisées. Puis dans un second temps, il va appliquer ces actions et récupérer les observations obtenus et les transmettre à chaque agent, ces dernier vont ensuite les ajoutés à leur historique.

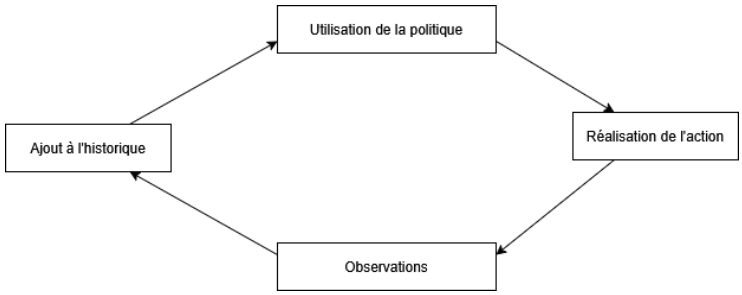


FIGURE 1 – Cycle du simulateur

3 Implémentations

3.1 Hanabi

3.1.1 Principe

Hanabi est un jeu de société coopératif dont l'objectif est pour chacune des 5 couleurs de placer les dans l'ordre de 1 à 5.

La particularité est que chaque joueur ne voit pas ses propres cartes mais, au contraire, voit celles des autres joueurs, à chaque tour un joueur à la possibilité de jouer une de ses cartes, d'utiliser un jeton d'indice afin d'indiquer de quoi est composé la main de ce dernier ou de défausser une carte, ce qui permet de récupérer un jeton indice.

Si un joueur joue une carte qui ne respecte pas les règles, c'est-à-dire qui ne permet ni d'augmenter une suite déjà présente, ni d'en créer une nouvelle, alors l'équipe reçoit un jeton d'erreur.

Le jeu s'arrête si l'équipe reçoit 3 jetons d'erreurs ou quand il n'y a plus de pioche. Dans le 1er cas, l'équipe a perdu et gagne donc 0 point, dans le second cas, l'équipe a gagné et emporte un nombre de point égal à la somme des points pour toutes les plus grandes pile de chaque couleur. Le score maximale est donc obtenu en ayant réussit une pile de chaque couleur allant jusqu'à cinq et le score associé est donc de 25.

Afin de réussir à être efficace, les joueurs doivent donc deviner, au travers des actions des autres joueurs, des informations qui ne sont pas directement explicités mais également réfléchir à ce que savent les autres joueurs afin d'éviter de leur donner un indice inutile.

3.1.2 Implémentation

Nous avons commencé par créer une classe Carte, chaque carte prenant un numéro et une couleur. Puis nous avons utilisé une Map<Agent, List<Carte» afin de représenter les mains des joueurs, la pioche est représenté par une List<Carte>, les piles de jeu sont représentées par une Map<String, Deque<Carte» qui associe à chaque couleur une pile de carte.

Nous avons également implémenté la boucle de jeu ainsi que les conditions de victoire dans JeuHanabi, mais nous n'avons pas encore mis en place un nombre limité d'indice.

3.2 Jeu de la ville

3.2.1 Principe

Le jeu de la ville est une simulation dans laquelle plusieurs individus doivent se retrouver dans une ville sans pouvoir communiquer entre eux.

Les agents possèdent plusieurs choix à chaque tour, aller à une case voisines, rester sur place, laisser un message sur la case actuelle.

De plus durant la simulation, la ville évolue, en effet certaines routes peuvent se retrouver bloqué ou à minima ralenti.

Pour pouvoir se retrouver les individus doivent donc essayer de deviner ou se trouve l'autre individu ainsi que les connaissances qu'il pourrait avoir, pour pouvoir s'adapter et agir en conséquence.

3.2.2 Implémentation

Pour l'implémentation du jeu de la ville, nous avons décidé de le modélisé par un graphe dans lequel chaque sommet représente un lieu-dit (Métro, Centre-ville ...) et chaque arête représente un chemin permettant d'aller d'un lieu à un autre.

Pour l'instant, nous devons construire la ville manuellement ainsi que décidé quelles routes sont bloquées mais nous avons pour objectif d'automatisé ces étapes ainsi que de rendre dynamique le blocage des routes, c'est-à-dire qu'une route puisse se bloquer en cours de simulation.

Nous allons également pondéré les arêtes du graphe afin de représenté le temps que prend le trajet d'un lieu à un autre. De plus, il nous suffira d'augmenter le poids de l'arête à une valeur très grande pour représenté le fait qu'elle soit bloqué.

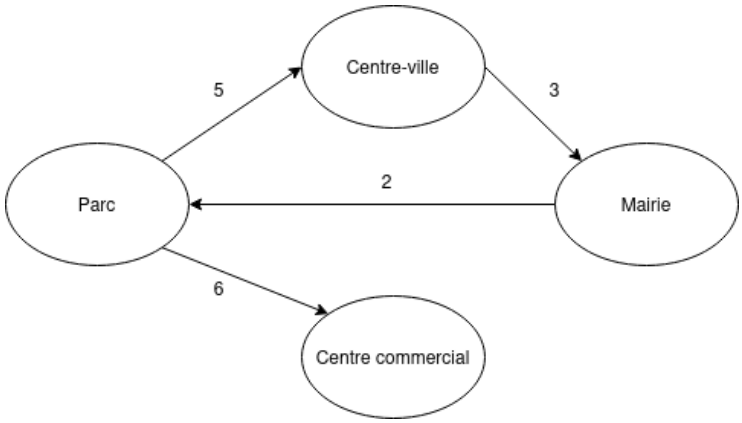


FIGURE 2 – Exemple de graphe du jeu de la ville

4 Conclusion

4.1 Avancé actuelle

Nous avons donc actuellement implémenté un système d'agent fonctionnel, robuste et adaptable à n'importe quel problème.

Nous avons également créer 2 implémentations distinctes et possédant des caractéristiques très différentes l'une de l'autre afin de montrer l'adaptabilité de notre agent.

La première étant le jeu de la ville, dans laquelle nous possédons une boucle de jeu fonctionnelle et dont il nous reste que quelques détails à régler.

La seconde étant Hanabi qui mise à part un système de jetons d'indices et entièrement fonctionnelle.

Nous n'avons cependant pas encore de politique adapté à chacune des implémentations. Nous utilisons donc une politique par défaut qui joue aléatoirement à chaque pas de temps.

4.2 Répartition des tâches

Tom David a implémenté l'agent puis s'est occupé d'implémenté le jeu de la ville avec Marius Guillais afin de pouvoir tester l'agent dans une réelle implémentation.

Titouan Pasquier s'est occupé d'implémenté Hanabi afin de pouvoir testé le caractère adaptatif de notre agent.

Marius Guillais a d'abord commencé par aidé Tom David sur l'implémentation du jeu de la ville, puis s'est occupé du système de message et d'observation, enfin il a implémenté un système d'historique car dans nos premières versions, nous utilisions seulement l'état du jeu ce qui permettait aux agents d'avoir des informations qu'ils n'étaient pas censés avoir.

Maxim Chepy a travaillé sur l'implémentation de politiques complexes qui n'ont pas pu être finalisé à temps avant de se concentrer sur la rédaction du rapport.

4.3 Améliorations

La prochaine étape pour nous est de finaliser les 2 implémentations (Jeu de la ville, Hanabi) mais également de nous intéressé à la mise en place de politiques complexes pour chacune des implémentations afin qu'ils puissent être utilisé par nos agents.

Nous pourrons par exemple réfléchir à des politiques composé d'un leader et de suiveurs, c'est-à-dire un agent qui fait les choix principaux et les autres qui s'adapte à ce dernier, mais également des politiques plus naïves ne prenant que très peu les choix des autres agents en compte, ou d'autre qui permettront de changer l'objectif en cours de simulation.

Le principal défi sera de réussir à faire en sorte que les politiques prennent en compte les décisions des autres agents afin de s'y adapter mais également de réussir à déduire des informations qui ne sont pourtant pas directement explicitées.